Chapter 36 - Keyboard, Mouse, and Time of Day

Input on the Intuition Engine is keyboard plus mouse, plus a small real-time clock. Every input register lives in the terminal / serial block (`0xF0700-0xF07FF`). Each is `32`-bit on the bus; only the documented low bits carry information.

This chapter documents the raw register interface. BASIC programs that want characters typically read them through `GET` and `INPUT` (Chapter 2) and never touch these registers directly. Games and graphic programs use them.

36.1 The keyboard

Two paths from the keyboard reach a running program: a cooked character stream (the same one BASIC uses) and a raw scancode stream.

36.1.1 Cooked keys

Address	Name	R/W	Meaning
`0xF0728`	raw-key dequeue	R	Read one raw key code, then advance the queue.
`0xF072C`	raw-key status	R	Bit `0` is set when at least one raw key is queued.

The "raw key" here is the cooked key character (already mapped from the keyboard layout, with the shift state applied, but not echoed and not line-buffered). A typical input loop:

```
.loop:
    load.l  r1, 0xF072C
    beqz    r1, .loop
    load.l  r2, 0xF0728
    ; R2 = key code
```

Reading `0xF0728` when no key is queued returns `0`. Reading the status first avoids losing track of the queue's empty state.

36.1.2 Raw scancodes

For programs that want the physical key, not its character (a game remap screen, a music tracker, an editor's keymap), the scancode path is the right one:

Address	Name	R/W	Meaning
`0xF0740`	scancode dequeue	R	Read one raw scancode, then advance the queue.
`0xF0744`	scancode status	R	Bit `0` is set when at least one scancode is queued.
`0xF0748`	modifier byte	R	Bit `0` = shift, bit `1` = ctrl, bit `2` = alt, bit `3` = capslock.

Scancodes are the keyboard's own numbering, not the cooked character. A scancode arrives once on press and once on release; press and release are distinguished by the high bit of the value (the high bit set means "release"). The modifier byte is a snapshot of the current modifier state at the moment of the read and is not queued; it reflects what is being held now.

36.2 The mouse

The mouse has two interfaces: an absolute one (where on the screen the cursor is) and a relative one (how far the mouse has moved since the last read).

36.2.1 Absolute mode

Address	Name	R/W	Meaning
`0xF0730`	mouse X	R	Absolute X in screen pixels, low `16` bits.
`0xF0734`	mouse Y	R	Absolute Y in screen pixels, low `16` bits.
`0xF0738`	mouse buttons	R	Bit `0` = left, bit `1` = right, bit `2` = middle.
`0xF073C`	mouse status	R	Bit `0` is set when the mouse state has changed since the last read of this register. Clears on read.

The status register's "changed" bit is a one-shot. A program that wants to know it has the latest position polls `0xF073C`, and on a non-zero result reads X, Y, and buttons.

36.2.2 Relative mode

For first-person 3-D games and pointer-locking work, the relative interface is the right one. It uses three registers:

Address	Name	R/W	Meaning
`0xF074C`	mouse control	W	Bit `0` = request relative / captured mouse mode.
`0xF0754`	accumulated dX	R	Signed accumulated relative X movement since the last read. Clears on read.
`0xF0758`	accumulated dY	R	Signed accumulated relative Y movement since the last read. Clears on read.

The program writes `1` to bit `0` of the control register to ask for relative mode. The cursor disappears and movement is reported through the dX / dY registers. Writing `0` returns to absolute mode and the cursor reappears at its last known position.

dX and dY are signed 32-bit values that accumulate between reads. A game loop that polls once per frame sees the total movement since the previous frame.

36.3 Time of day

A single register reads back the wall-clock time:

Address	Name	R/W	Meaning
`0xF0750`	epoch time	R	Seconds since the Unix epoch (1970-01-01 00:00:00 UTC), as a signed `32`-bit count.

The register is updated continuously; two reads taken a second apart differ by one. There is no separate sub-second register and no per-CPU latch. Programs that want a finer granularity should combine this with a CPU timer (Chapter 30).

The value rolls over to a negative number in the year 2038, after which it counts back toward zero, and through zero into positive territory again, like any signed 32-bit second counter.

36.4 An input loop

A polling game loop that uses keyboard and relative mouse:

```
.frame:
    ; Drain key queue
.key_loop:
    load.l  r1, 0xF072C
    beqz    r1, .key_done
    load.l  r2, 0xF0728
    jsr     handle_key
    bra     .key_loop
.key_done:

    ; Read accumulated mouse motion
    load.l  r1, 0xF0754   ; signed dX
    load.l  r2, 0xF0758   ; signed dY
    jsr     handle_motion

    ; ... rest of frame ...
    bra     .frame
```

The key drain is unbounded: the program keeps consuming until the status register reads `0`. The mouse reads happen exactly once per frame because the dX / dY registers clear themselves on read.

36.5 Use from the small CPUs

The 6502 and Z80 reach every register here through the same bank-window mechanism that exposes the rest of the `0xF0000-0xFFFFF` region. Each `32`-bit register is read by the 8-bit CPUs as four byte-aligned bytes; the meaningful bits sit in the low byte of every register except the mouse position registers (which use bits `0-15` and so span the low two bytes). The bank-window scheme and exact 16-bit addresses are described in Chapters 26 and 27.

The cooked-key path at `0xF0728` is also the path that the terminal uses (Chapter 37). A program that calls `INPUT` from BASIC, or that reads the terminal input register from machine code, is competing with `0xF0728` for the same queue; pick one consumer per key stream.